

Περίληψη μαθήματος για πρωτοετείς

Πολυμορφισμός, υπερφόρτωση μεθόδων, υπερκάλυψη, toString(), super και equals()
στη Java

Πηγή: σύνοψη βασισμένη στις διαφάνειες του αρχείου 02-override.pdf.

1. Τι είναι πολυμορφισμός;

Πολυμορφισμός σημαίνει ότι ο ίδιος κώδικας μπορεί να δουλεύει με αντικείμενα διαφορετικών τύπων, αρκεί να έχουν κοινό τρόπο χρήσης. Στην πράξη, γράφουμε πιο γενικό κώδικα και αφήνουμε τη Java να επιλέξει τη σωστή συμπεριφορά.

Μια απλή ιδέα είναι η κατασκευή String:

```
int i = 10;
LocalDate d = new LocalDate(2000, Month.JANUARY, 1);
System.out.println("There are " + i + " elements at " + d);
```

- Το + μπορεί να συνδυάσει String με int, ημερομηνία ή άλλο αντικείμενο.
- Για τα αντικείμενα, η Java χρειάζεται μια αναπαράσταση σε κείμενο, συνήθως μέσω της toString().
- Αυτό δείχνει ότι μια πράξη μπορεί να έχει διαφορετική συμπεριφορά ανάλογα με τους τύπους των τιμών.

2. Υπερφόρτωση μεθόδων (method overloading)

Υπερφόρτωση έχουμε όταν πολλές μέθοδοι έχουν το ίδιο όνομα, αλλά διαφορετική λίστα παραμέτρων. Ο μεταγλωττιστής επιλέγει ποια θα καλέσει με βάση τα ορίσματα που δίνουμε.

```
class StringAppender {
    void append(char c) { ... }
    void append(char[] chars) { ... }
    void append(char[] chars, int start, int length) { ... }
    void append(String s) { ... }
}
```

Παράδειγμα χρήσης:

```
char[] buffer = new char[100];
reader.read(buffer);
appender.append(buffer);           // όλος ο πίνακας χαρακτήρων
appender.append(buffer, 10, 20);  // τμήμα του πίνακα
```

3. Καλή σχεδίαση: μία βασική υλοποίηση και οι άλλες να την καλούν

Συχνά υλοποιούμε πρώτα την πιο βασική μορφή και μετά οι άλλες υπερφορτωμένες μέθοδοι τη χρησιμοποιούν. Έτσι αποφεύγουμε διπλό κώδικα.

```
class StringAppender {
    void append(char c) { ... }

    void append(char[] chars) {
        append(chars, 0, chars.length);
    }

    void append(char[] chars, int start, int length) {
        for (int i = start; i < start + length; i++) {
            append(chars[i]);
        }
    }

    void append(String s) {
        append(s.toCharArray());
    }
}
```

- Αν αλλάξει η αποθήκευση χαρακτήρων, αλλάζουμε κυρίως την `append(char c)`.
- Οι άλλες μέθοδοι εκφράζουν ευκολίες χρήσης, όχι ξεχωριστή λογική.

4. Το πρόβλημα: δεν γίνεται να γράφουμε `append` για κάθε τύπο

Αν θέλουμε να προσθέτουμε `Integer`, `LocalDate`, `User`, `PowerUser` κ.λπ., δεν είναι καλή λύση να γράφουμε συνεχώς νέα `append` για κάθε κλάση.

```
class StringAppender {
    public void append(Integer i) { ... }
    public void append(LocalDate d) { ... }
    public void append(User u) { ... }
    // και μετά PowerUser, Guest, AdminUser, ...
}
```

Η καλύτερη ιδέα είναι να χρησιμοποιήσουμε κοινό υπερτύπο. Στη Java κάθε κλάση κληρονομεί, άμεσα ή έμμεσα, από την `Object`. Άρα μπορούμε να δεχτούμε `Object`:

```
class StringAppender {
    void append(Object o) {
        append(o.toString());
    }
}
```

- Η `append(Object)` μπορεί να δεχτεί οποιοδήποτε αντικείμενο.
- Η `toString()` μετατρέπει το αντικείμενο σε κείμενο.
- Αν μια κλάση θέλει καλύτερη εμφάνιση, υπερκαλύπτει την `toString()`.

5. Πολυμορφικές μεταβλητές

Μια μεταβλητή τύπου γονικής κλάσης μπορεί να δείχνει σε αντικείμενο υποκλάσης. Αυτό είναι βασικό κομμάτι του πολυμορφισμού.

```
User u;
u = new PowerUser();
u = new Guest();
```

- Ο δηλωμένος τύπος της μεταβλητής είναι `User`.

- Το πραγματικό αντικείμενο μπορεί να είναι PowerUser ή Guest.
- Έτσι γράφουμε κώδικα που δουλεύει γενικά με χρήστες, χωρίς να ξέρει κάθε ειδική κατηγορία χρήστη.

6. Υπερ κάλυψη μεθόδων (method overriding)

Υπερ κάλυψη έχουμε όταν μια υποκλάση ορίζει ξανά μέθοδο που υπάρχει στη γονική κλάση, με την ίδια υπογραφή. Τότε, όταν καλείται η μέθοδος σε αντικείμενο, εκτελείται η έκδοση που αντιστοιχεί στον πραγματικό τύπο του αντικειμένου.

```
class Animal {  
    void speak() { }  
}  
class Cat extends Animal {  
    void speak() { System.out.println("meow"); }  
}  
class Dog extends Animal {  
    void speak() { System.out.println("woof"); }  
}
```

7. Υπερκάλυψη της toString()

Η `toString()` είναι μέθοδος της `Object`. Από προεπιλογή δεν δίνει πάντα χρήσιμο κείμενο για τον άνθρωπο. Για αυτό συχνά την υπερκαλύπτουμε.

```
class User {  
    private String username;  
  
    public String toString() {  
        return username;  
    }  
}
```

- Όταν κάνουμε `System.out.println(user)`, χρησιμοποιείται η `toString()`.
- Όταν κάνουμε `append(Object)`, μπορούμε εσωτερικά να καλέσουμε `o.toString()`.
- Έτσι κάθε κλάση αποφασίζει μόνη της πώς θα εμφανίζεται ως κείμενο.

8. Χρήση της super

Όταν μια υποκλάση υπερκαλύπτει μια μέθοδο, μερικές φορές θέλουμε να κρατήσουμε τη συμπεριφορά της γονικής κλάσης και απλώς να προσθέσουμε κάτι. Τότε χρησιμοποιούμε `super`.

```
class AdminUser extends User {  
    public void login() {  
        super.login();  
        System.out.println("User " + getName() + " logged in as admin");  
    }  
}
```

- Το `super.login()` καλεί την `login()` της `User`.
- Μετά η `AdminUser` προσθέτει τη δική της ειδική συμπεριφορά.

Παράδειγμα με επιστρεφόμενη τιμή:

```
class PowerUser extends User {  
    public int getQuota() {  
        return (int)(super.getQuota() * 1.2);  
    }  
}
```

- Η `PowerUser` παίρνει το `quota` της `User` και το αυξάνει κατά 20%.

9. Υπερφόρτωση vs υπερκάλυψη

Έννοια	Υπερφόρτωση	Υπερκάλυψη
Πού συμβαίνει;	Στην ίδια κλάση ή και σε συγγενικές κλάσεις.	Σε υποκλάση που κληρονομεί μέθοδο.
Υπογραφή;	Ίδιο όνομα, διαφορετικές παράμετροι.	Ίδιο όνομα και ίδιες παράμετροι.
Πότε επιλέγεται;	Κατά τη μεταγλώττιση, με βάση τα ορίσματα.	Κατά την εκτέλεση, με βάση το πραγματικό αντικείμενο.
Παράδειγμα	<code>append(String)</code> , <code>append(char[])</code>	<code>Cat.speak()</code> αντί <code>Animal.speak()</code>

10. Ισότητα αντικειμένων: == και equals()

Το == στα αντικείμενα δεν συγκρίνει το περιεχόμενο. Συγκρίνει αν δύο μεταβλητές δείχνουν στο ίδιο ακριβώς αντικείμενο στη μνήμη.

```
User u1 = new User("user1");
User u2 = new User("user1");
```

```
if (u1 == u2) {
    u2.logout();
}
```

- Τα u1 και u2 έχουν ίδιο username.
- Όμως είναι δύο διαφορετικά αντικείμενα. Άρα u1 == u2 είναι false.

```
public boolean equals(Object x) { ... }
```

- Η παράμετρος είναι Object, επειδή η equals πρέπει να μπορεί να συγκρίνει με οποιοδήποτε αντικείμενο.
- Η κλάση αποφασίζει ποια πεδία μετράνε για την ισότητα. Για User μπορεί να αρκεί το username και να αγνοείται το lastLogin.

11. Λάθος πρώτη προσπάθεια στην equals()

Ένας αρχάριος συχνά γράφει κάτι σαν αυτό:

```
class User {
    private String username;

    public boolean equals(Object o) {
        return username.equals(o.username); // λάθος
    }
}
```

Το λάθος είναι ότι η παράμετρος o έχει δηλωμένο τύπο Object. Ο τύπος Object δεν έχει πεδίο username, άρα ο κώδικας δεν μεταγλωττίζεται.

Επίσης, μπορεί κάποιος να καλέσει:

```
User u = new User("user1");
u.equals("user1");           // πρέπει να επιστρέψει false
u.equals(new Integer(10));   // πρέπει να επιστρέψει false
u.equals(new PowerUser("user1")); // μπορεί να επιστρέψει true, ανάλογα με τον κανόνα
```

12. Σωστή βασική μορφή equals() με instanceof και cast

Πρώτα ελέγχουμε αν το αντικείμενο είναι User. Μετά το μετατρέπουμε σε User, ώστε να μπορούμε να διαβάσουμε το username.

```
class User {
    private String username;
    private LocalDateTime lastLogin;

    public boolean equals(Object o) {
        if (o instanceof User) {
            User u = (User) o;
            return username.equals(u.username);
        } else {
            return false;
        }
    }
}
```

- Το instanceof προστατεύει από λανθασμένους τύπους.
- Το cast (User) ο λέει στη Java ότι από εδώ και πέρα θα το χειριστούμε ως User.
- Αν δεν είχαμε κάνει έλεγχο, ένα λάθος cast θα μπορούσε να οδηγήσει σε ClassCastException.

13. Ισότητα σε υποκλάσεις: προσοχή στον κανόνα

Μια υποκλάση μπορεί να θέλει αυστηρότερο κανόνα ισότητας. Για παράδειγμα, ένας AdminUser μπορεί να θεωρείται ίσος μόνο με άλλο AdminUser.

```
class AdminUser extends User {
    public boolean equals(Object o) {
        if (o instanceof AdminUser) {
            return super.equals(o);
        } else {
            return false;
        }
    }
}
```